



Hush

A simple env manager and vault for project secrets. Project documentation and presentation guide.

Friday Godswill Essien (23/SC/CO/158)

Department of Computer Science, University of Uyo

300 level project, October 2026

Live: <https://hush-production-70be.up.railway.app>

Demo login: demo@hush.ng / demo1234

Contents

1. Overview
2. How the core logic works
3. Architecture and data model
4. Walkthrough of every screen
5. Mobile view
6. Running locally
7. Testing
8. Deployment
9. Five-minute presentation script

1. Overview

Hush is a web application for keeping the environment variables of software projects in one safe place. Developers usually keep database passwords, payment keys and API tokens in `.env` files that get copied between laptops, pasted into group chats and sometimes committed to GitHub by mistake. Hush replaces that habit with an encrypted vault that also tells you which keys are old, which are reused, and who looked at what.

The problem

Student and startup teams in Nigeria often share secrets on WhatsApp, keep the same password in development and production, and never rotate keys after a teammate leaves. When a key leaks, nobody knows how old it is or where else it is used.

What Hush does

Feature	What it gives the user
Encrypted vault	Projects with development, staging and production variables. Values are encrypted with AES-256-GCM before they reach the database.
Rotation tracking	Each secret has a rotation period and an optional expiry. Hush marks it healthy, due soon, overdue or expired, and scores the whole vault.
Reuse detection	A keyed fingerprint of each value shows when the same secret is used in two places, without comparing plaintext.
.env import and export	Paste a .env file to import it, or download an environment as a .env file. Bad lines are reported with line numbers.
One-time share links	Send a secret with a link that works a set number of times and expires. The server keeps only a hash of the link.
Leak scanner	Paste code or logs to find API keys, tokens, private keys and passwords. Runs in the browser.
Image scrambler	Scramble a picture with a key so only people with the key can restore it.
File sealing	Encrypt any file with a passphrase in the browser and download a .hush file.
Secret generator	Strong random values with an entropy estimate.
Activity log	Every sign-in, failed sign-in, reveal, export and share is recorded with the IP address.

Demo account

The live site is **<https://hush-production-70be.up.railway.app>**. The sign-in page is pre-filled with `demo@hush.ng` and `demo1234`. The demo vault has three projects with a mix of healthy, due, overdue and expired secrets, share links in every state and two weeks of activity, so every screen has something to show.

Technology

Layer	Choice
Framework	Next.js 16 (App Router, Server Components, Server Actions), React 19, TypeScript in strict mode
Database	PostgreSQL with Drizzle ORM and versioned SQL migrations from drizzle-kit
Validation	Zod schemas with one inline error per field
Authentication	bcryptjs password hashes, HS256 JWT signed with jose in an HTTP-only cookie
Cryptography	Node crypto (AES-256-GCM, HKDF, SHA-256) on the server; Web Crypto (PBKDF2, AES-GCM) in the browser
Interface	Plain CSS in a monochrome dark theme, lucide-react icons, DiceBear avatars made locally, Inter and JetBrains Mono from Fontsource
Tests and hosting	Vitest, Playwright, GitHub and Railway

2. How the core logic works

All business rules live as pure functions in `src/lib`. They take "today" or "now" as an argument instead of reading the clock, so tests and demos are repeatable. Setting `HUSH_TODAY=YYYY-MM-DD` pins the date for the whole app.

Encryption at rest

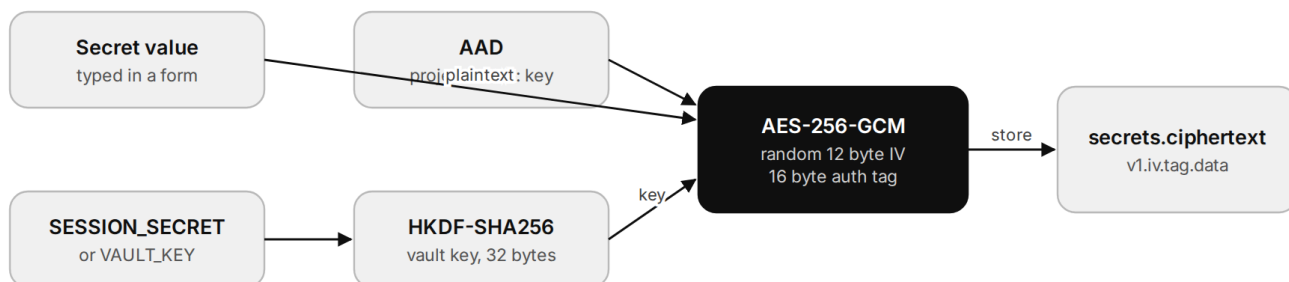
The vault key is derived from `SESSION_SECRET` (or a separate `VAULT_KEY`) with HKDF-SHA256 and a purpose label, so the session key and the vault key are different even though they come from one secret. Each value is encrypted with AES-256-GCM using a fresh random 12-byte IV. The stored text is `v1.iv.tag.data`.

The encryption also takes additional authenticated data (AAD) made of the project id, environment and key name. If someone with database access copies the ciphertext of a production key into a staging row, decryption fails because the AAD no longer matches. Changing any byte fails the 16-byte authentication tag.

```
export function encrypt(plaintext: string, key: Buffer, aad = ""): string {
  const iv = randomBytes(12);
  const cipher = createCipheriv("aes-256-gcm", key, iv);
  cipher.setAAD(Buffer.from(aad));
  const ct = Buffer.concat([cipher.update(plaintext, "utf8"), cipher.final()]);
  return ["v1", iv, cipher.getAuthTag(), ct].map(b64url).join(".");
}
```

Rotation status

`rotationStatus(secret, today)` computes the due date as last rotated plus the rotation period, then checks in order: expired (expiry date before today), overdue (due date before today), due (within 7 days) and otherwise ok. Saving a new value is treated as a rotation: the version number goes up and the clock restarts. The vault health score starts at 100 and loses points per secret, with expired costing more than overdue, and overdue more than due.



Moving a ciphertext to another row changes the AAD, so decryption fails. Changing one byte fails the tag check.

Figure 1. How a secret value is encrypted before it is stored

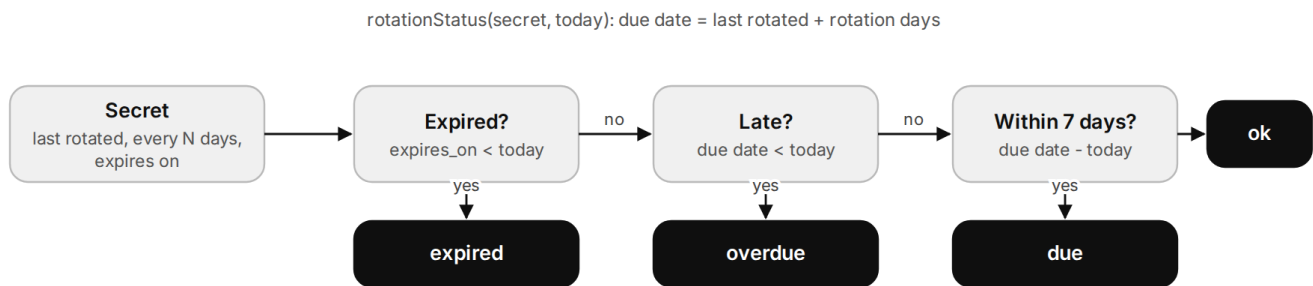


Figure 2. How the rotation status of a secret is decided

One-time share links

A link carries a random 24-byte token. The database stores only `sha256(token)` and a copy of the value encrypted with a key derived from the token itself, so even a full database leak does not reveal shared values. Opening the page only shows the label. The Reveal button runs a single conditional UPDATE that increases the view count only if the link is not revoked, not expired and has views left. Two people pressing Reveal at the same moment cannot both succeed. When the last view is used, the stored ciphertext is set to NULL.

Sign-in protection

Failed sign-ins are recorded in the audit log. `lockoutDecision(failures, now)` refuses sign-in after five failures for the same email in 15 minutes, even with the right password, until the oldest failure leaves the window. Unknown emails still run a `bcrypt` comparison so response times do not reveal which emails exist.

Leak scanner

`scanText` first applies rules for known formats (AWS access keys, GitHub tokens, Stripe and Paystack keys, Slack tokens, Google API keys, JWTs, private key blocks and database URLs with passwords). It then looks for assignments such as `API_TOKEN = "..."` and keeps only values whose Shannon entropy is at least 3 bits per character, which filters out placeholders like `changeme`. Matches are redacted before they are shown.

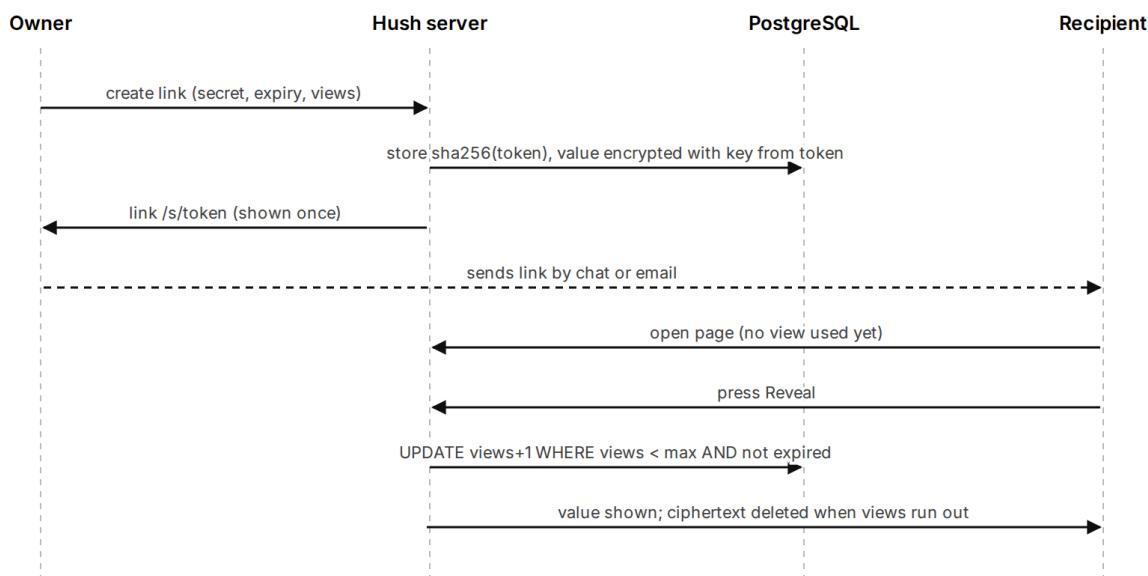


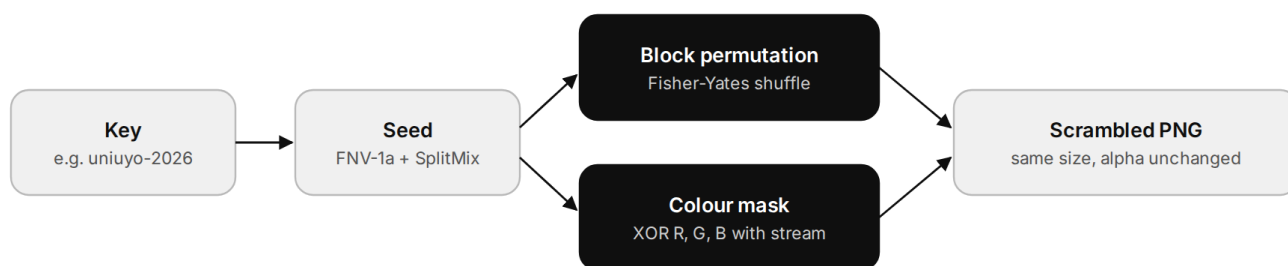
Figure 3. Creating and opening a one-time share link

Image scrambling

The key is hashed into a 32-bit seed. A deterministic generator seeded with it drives a Fisher-Yates shuffle of the image's blocks and produces a mask that is XOR-ed into the red, green and blue values of every pixel. Unscrambling applies the mask again and moves each block back. The result is exact for PNG files. This is keyed obfuscation that keeps the file a valid image; for real confidentiality Hush offers file sealing.

File sealing

The browser derives an AES-256 key from the passphrase with PBKDF2-SHA256 (210,000 iterations and a random 16-byte salt) and encrypts the original name, type and contents with AES-GCM. The output starts with the bytes HUSH1 so the tool can tell sealed files apart. A wrong passphrase or any change to the file fails the tag check.



Unscramble runs the same steps in reverse order with the same key and block size.

Figure 4. Image scrambling with a key

3. Architecture and data model

Hush is one Next.js application. `src/proxy.ts` runs before every page and redirects visitors without a valid session cookie to the sign-in page. Pages are Server Components that read from the database through services in `src/server`. Forms call Server Actions, which validate input with Zod and return field errors for the form to show inline. The scanner, scrambler, sealer and generator are Client Components that use only `src/lib` and browser APIs, so their input never leaves the device.

Folder	Contents
<code>src/lib</code>	Pure rules: crypto, rotation, envfile, scanner, shares, lockout, scramble, seal, generator, validation, session
<code>src/server</code>	Database services: vault (projects and secrets), shares, users, audit, seed, session helpers
<code>src/app</code>	Pages, layouts, Server Actions, the export route and <code>/api/health</code>
<code>src/db</code>	Drizzle schema and the connection pool
<code>drizzle</code>	Numbered SQL migrations
<code>scripts</code>	migrate, seed and the documentation builder
<code>tests</code>	unit, integration and e2e suites

Security measures

Threat	Measure
Database leak	Values encrypted with AES-256-GCM; share links stored as hashes; passwords hashed with bcrypt
Ciphertext swapping	AAD binds each value to its project, environment and key
Session theft by scripts	HTTP-only, SameSite=Lax cookie, Secure in production
Brute-force sign-in	Lockout after 5 failures in 15 minutes; failures logged with IP
Seeing other users' data	Every query filters by the signed-in user's id; other ids return 404
Link previews burning a share	Opening the page does not count; only the Reveal button does
Clickjacking and sniffing	X-Frame-Options DENY, nosniff, no-referrer, HSTS headers

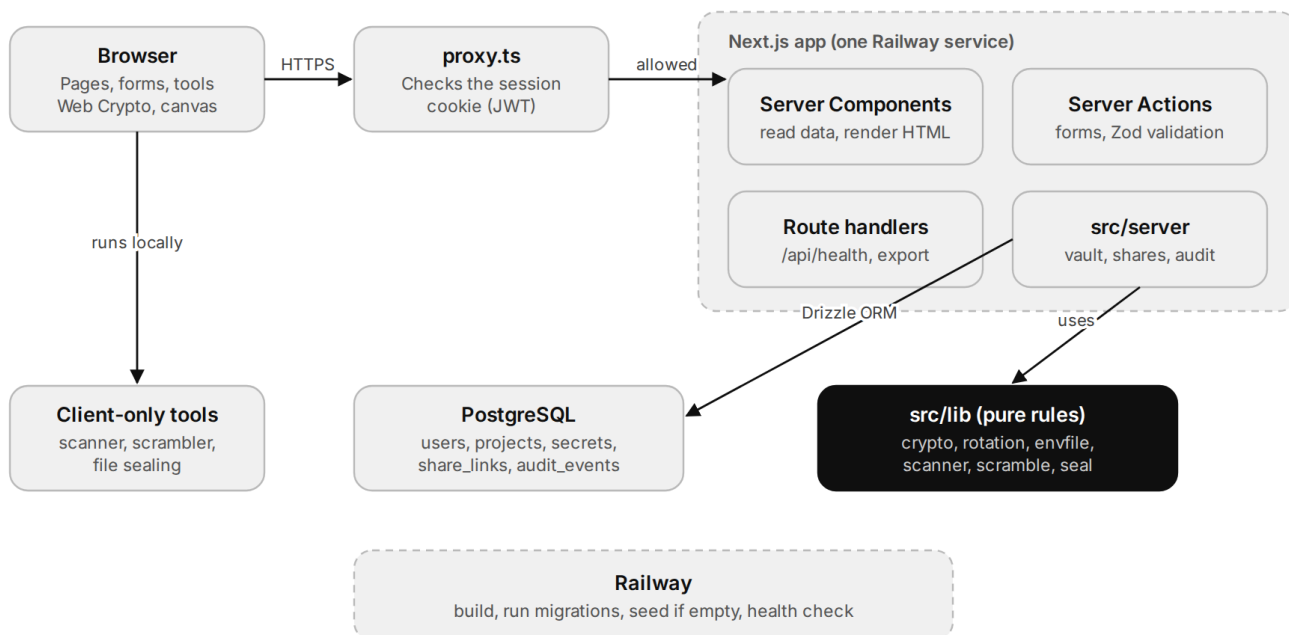


Figure 5. System architecture

The database has five tables. A unique index on (project_id, environment, key) stops duplicate keys even if application code has a bug. Deleting a project deletes its secrets through a cascading foreign key; deleting a secret keeps its share links for the audit trail but clears the link to the secret.

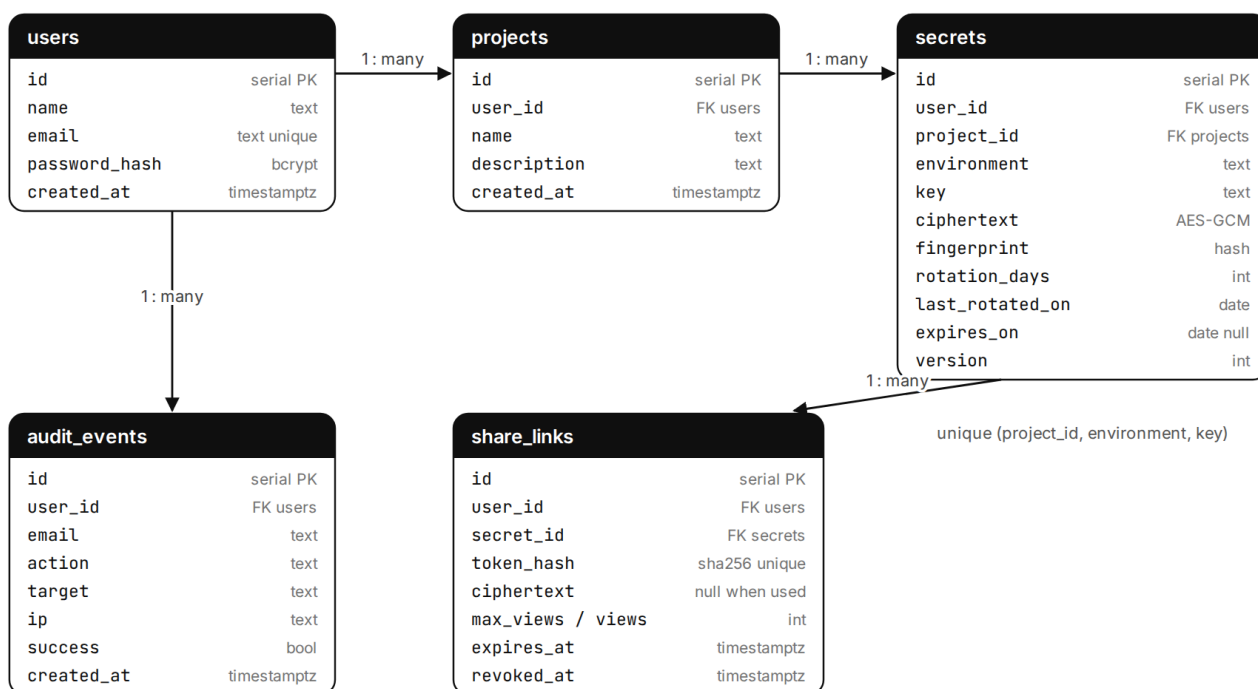


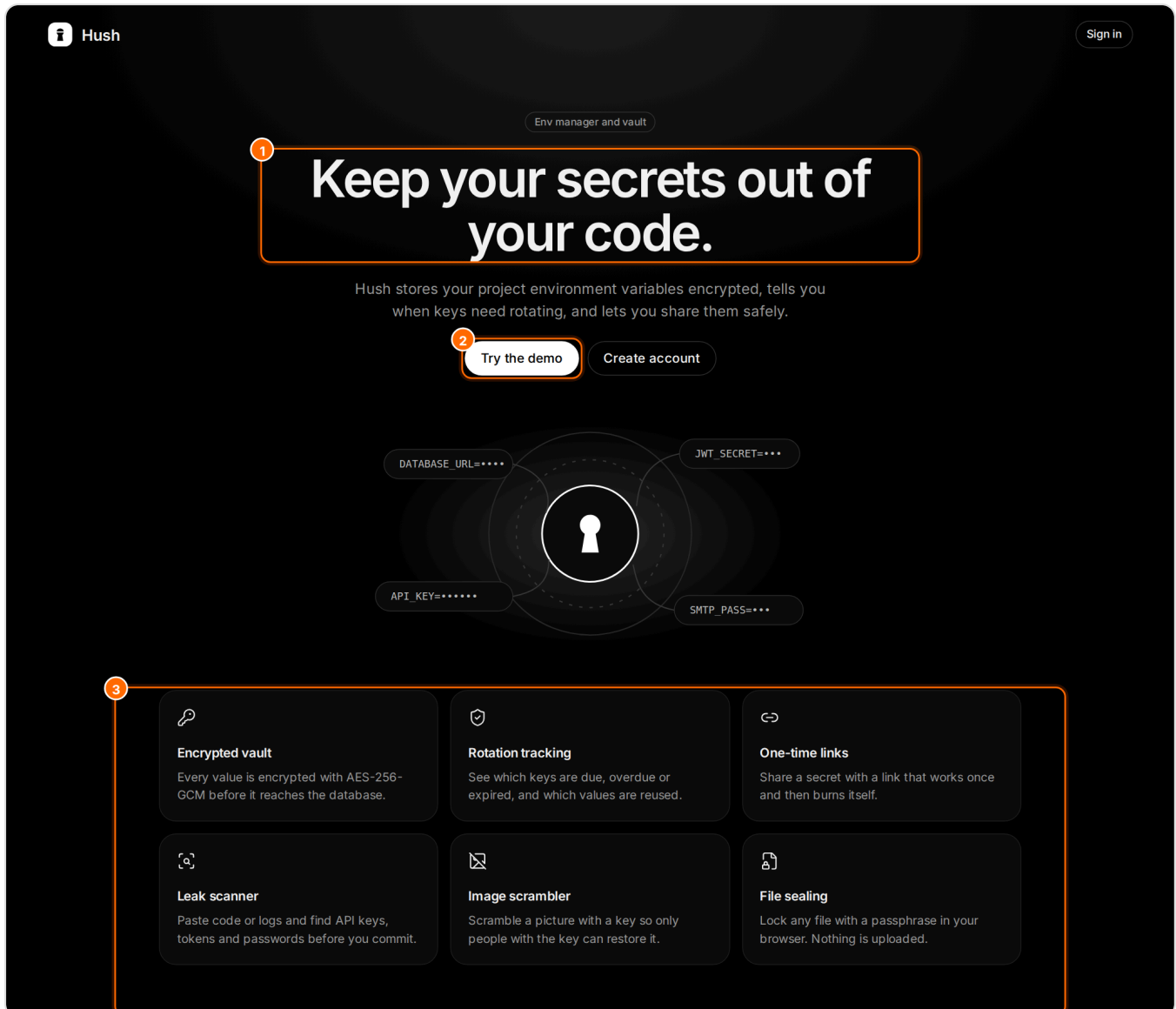
Figure 6. Entity relationship diagram

4. Walkthrough of every screen

Numbered orange markers on each screenshot match the numbered notes under it. Screenshots were taken from the seeded demo data with the date pinned to October 2026.

1. Landing page

The public home page explains what Hush does and sends visitors to the demo.



- 1 Headline that states the purpose in one line.
- 2 Try the demo opens the sign-in page with the demo account filled in.
- 3 Six feature cards: vault, rotation, one-time links, scanner, scrambler and file sealing.

2. Sign in

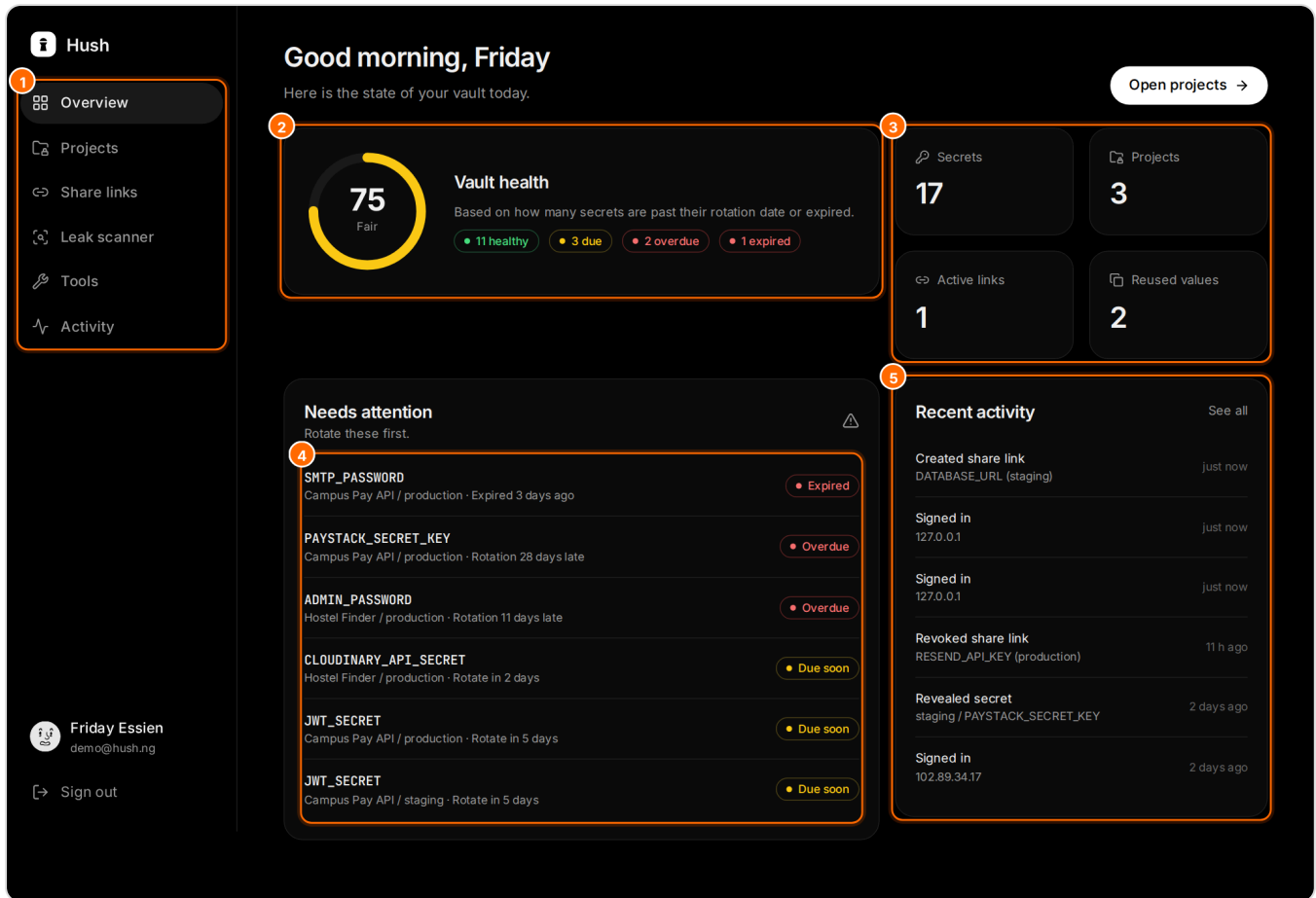
Email and password sign-in. After five failed attempts in 15 minutes the account is locked for the rest of the window.

The screenshot shows the Hush sign-in page. At the top, the Hush logo is displayed. Below it, the text 'Welcome back' and 'Sign in to open your vault.' are shown. The sign-in form consists of an 'Email' field with the value 'demo@hush.ng', a 'Password' field with masked characters, and a 'Sign in' button. A link for 'New here? Create an account' is located below the button. At the bottom, a message states 'Demo account is filled in: demo@hush.ng / demo1234'. Three orange numbered circles highlight specific elements: 1 points to the email field, 2 points to the 'Sign in' button, and 3 points to the demo account reminder message.

- 1 The demo email and password are pre-filled so the presenter only presses Sign in.
- 2 Submits a Server Action. Errors appear inline under each field.
- 3 Reminder of the demo credentials.

3. Overview (dashboard)

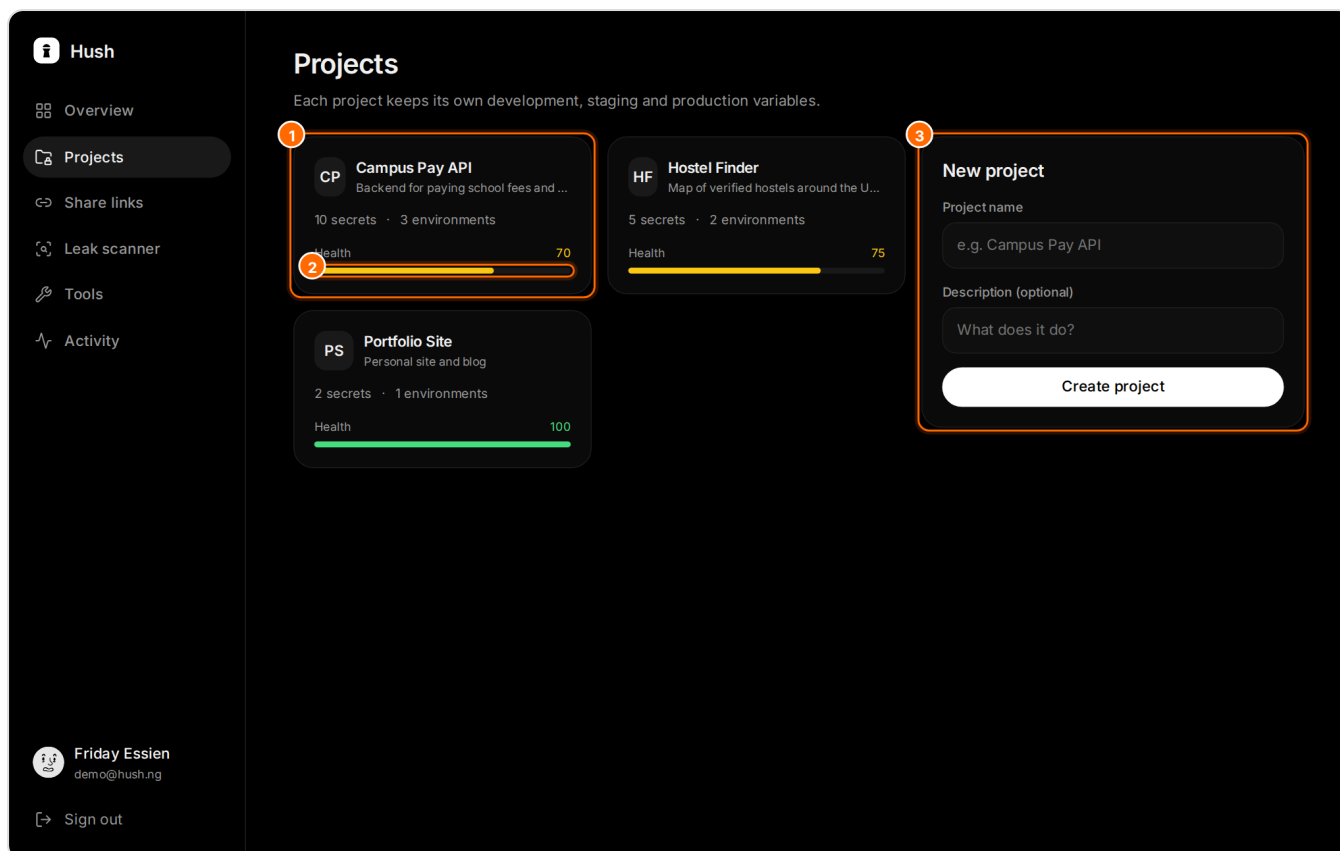
The first screen after sign-in summarises the health of the whole vault.



- 1 Main navigation. On phones it becomes a bottom tab bar.
- 2 Health score from 0 to 100 with counts of healthy, due, overdue and expired secrets.
- 3 Totals, active share links, and values that are reused in more than one place.
- 4 Secrets to rotate first, worst first. Each row opens the secret.
- 5 Latest audit events, including the failed sign-ins from an unknown IP.

4. Projects

Each project groups the variables of one application.



- 1 Project card with the number of secrets and environments.
- 2 Health bar for the project, coloured by score.
- 3 Create a project. Duplicate names are refused with an inline error.

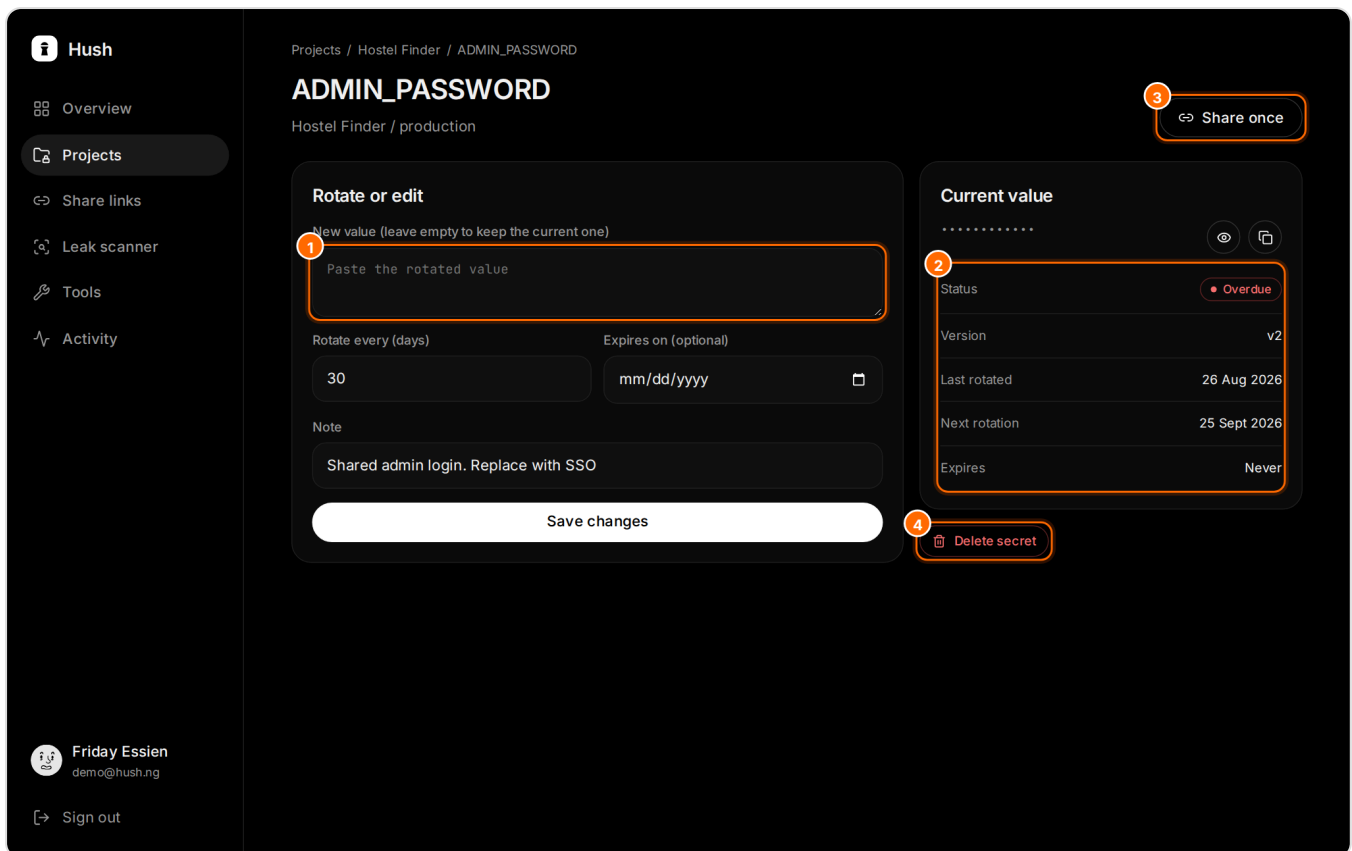
5. Project and its secrets

The main working screen. Values stay masked until revealed, and every reveal is logged.

- 1 Environment tabs with the number of variables in each.
- 2 Downloads the environment as a .env file. The export is logged.
- 3 A revealed value. Eye hides it again; the copy button copies without showing it.
- 4 Reused: the same value is stored under another key, here production and staging.
- 5 Rotation status with a plain explanation underneath.
- 6 Add a secret. The value is encrypted before it is saved.
- 7 Keys that exist in one environment but not the other.

6. Rotate or edit a secret

Opening a secret shows its rotation history and lets you save a new value.



- 1 A new value counts as a rotation: version goes up and the clock restarts today.
- 2 Status, version, last rotation, next rotation and expiry.
- 3 Opens Share links with this secret already selected.
- 4 Deletes the secret. The deletion is logged.

7. Share links

One-time links for sending a secret to a teammate without pasting it into a chat.

Share links

Send a secret with a link that stops working after it is opened or when it expires.

Your links 5 total

- DATABASE_URL (staging)**
Created 6 Oct, 09:00 · opened 0 of 1 · 32 h left Active Revoke
- DATABASE_URL (staging)**
Created 6 Oct, 01:04 · opened 0 of 1 · 24 h left Active Revoke
- RESEND_API_KEY (production)**
Created 5 Oct, 09:00 · opened 0 of 1 Revoked
- MAPBOX_TOKEN (production)**
Created 5 Oct, 05:00 · opened 1 of 1 Used
- PAYSTACK_SECRET_KEY (staging)**
Created 3 Oct, 03:00 · opened 1 of 3 Expired

New link

The link is shown once. Hush only keeps a hash of it.

Link created. Copy it now. It will not be shown again.

1 `http://localhost:3200/s/x8q7L0JWU8UpjKsuiTr4p0Puu9ZCxEx` Copy

Secret

Pick a secret ▼

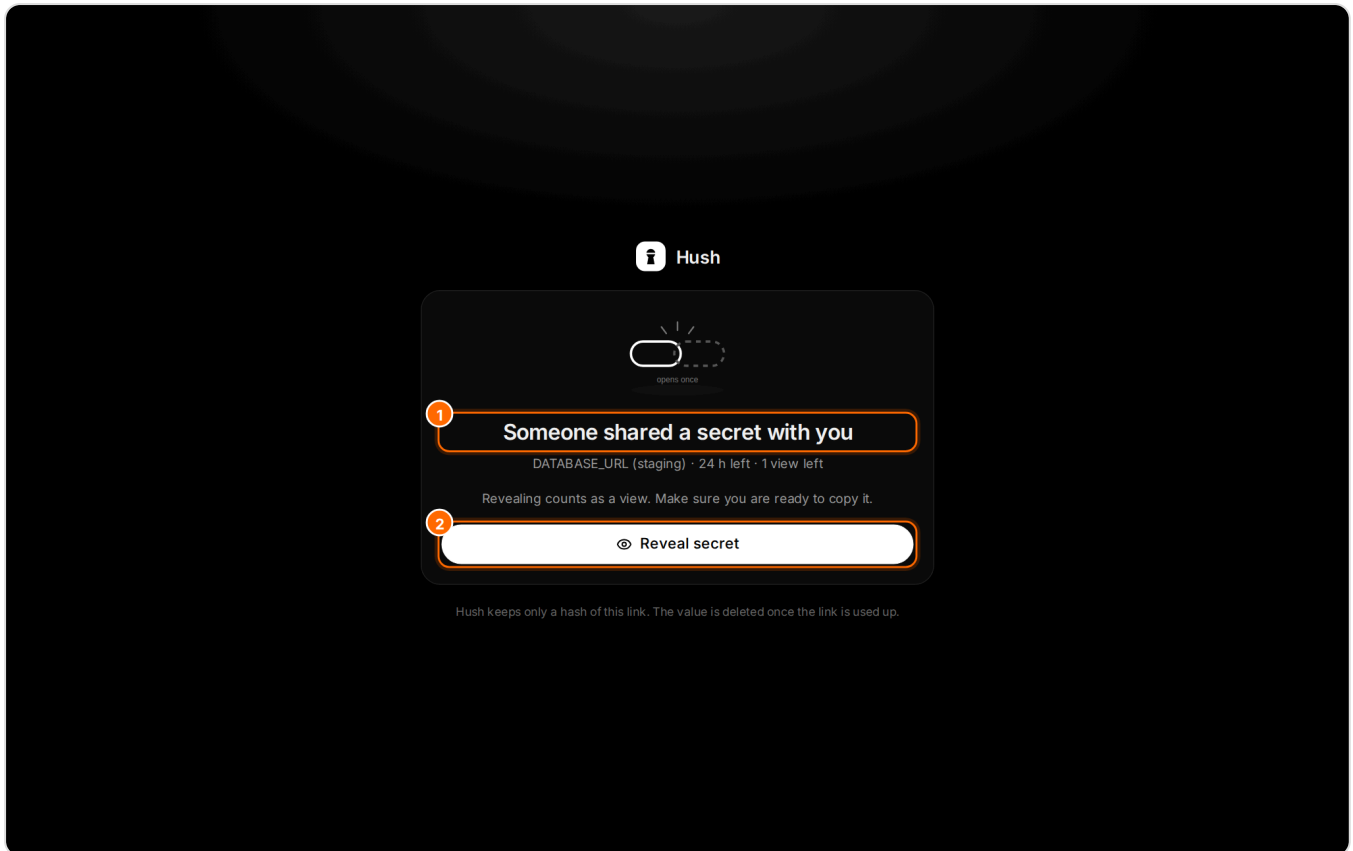
2 Expires after: 24 hours ▼ Can be opened: Once ▼

Create link

- 1 The new link is shown once. Hush stores only its SHA-256 hash.
- 2 Expiry (1 hour to 7 days) and how many times it can be opened.
- 3 Every link with its state: active, used, expired or revoked.
- 4 Revoke stops an active link and deletes its stored value.

8. Opening a shared link

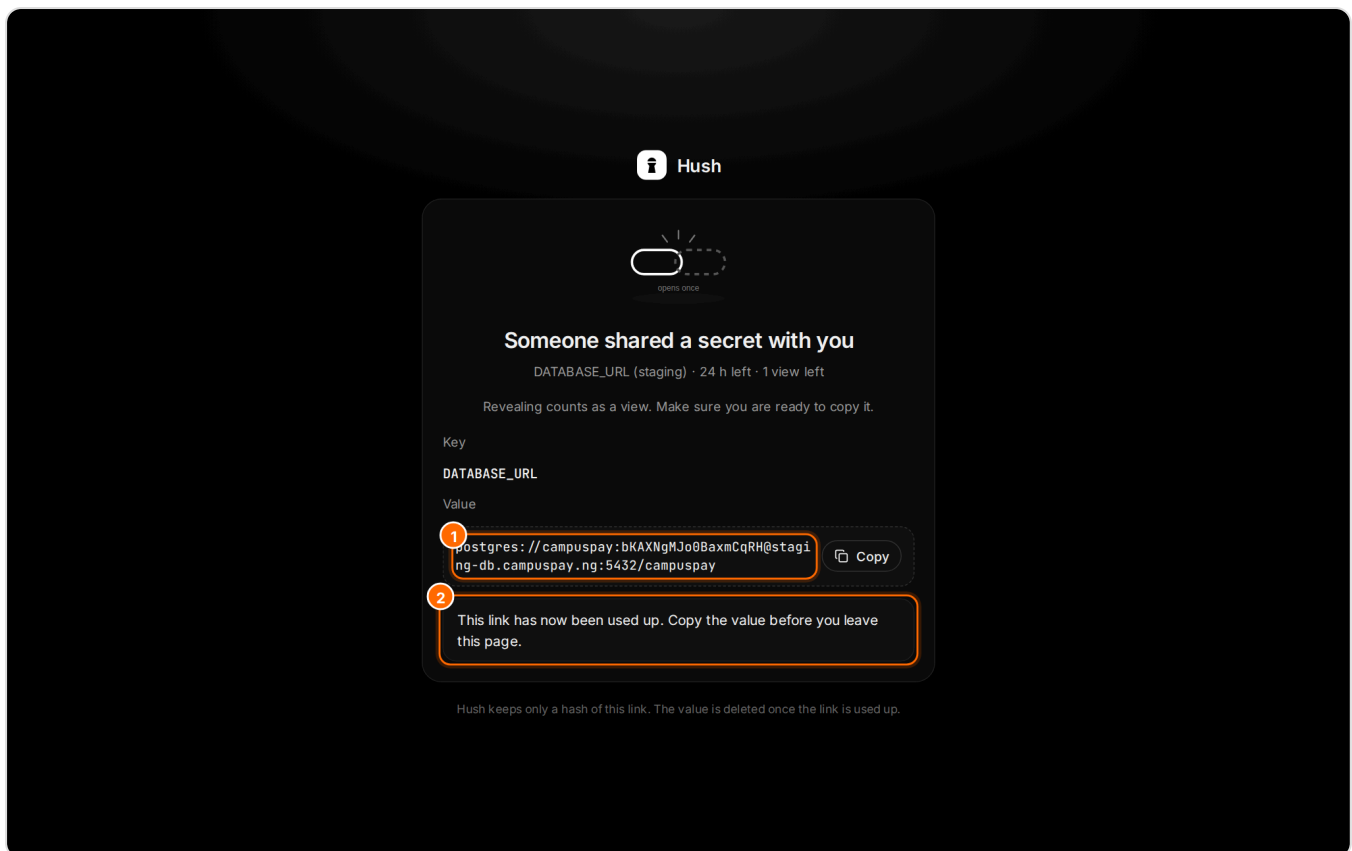
What the recipient sees. Opening the page does not use a view, so link previews in chat apps cannot burn it.



- 1 Who sent it is not shown. The label, time left and views left are.
- 2 Only this button uses a view.

9. Shared value revealed

After Reveal the value is shown once with a copy button, and the server deletes its copy.



- 1 The decrypted value with a copy button.
- 2 Tells the recipient the link is now used up.

10. Leak scanner

Finds credentials in pasted code or logs. It runs in the browser, so the text is never uploaded.

Hush

- Overview
- Projects
- Share links
- Leak scanner**
- Tools
- Activity

Leak scanner

Find API keys, tokens and passwords before they end up in Git or a group chat.

Paste code, logs or a message

Scanning happens in your browser. Nothing is sent to the server.

```
// payments.js
import axios from "axios";

const PAYSTACK_SECRET_KEY =
  "sk_live_ccdbf97233c5fe50679e843d9e6853d3ffe7a";
const awsKey = "AKIAMK4LL7XQN2B5UMKB";
const db =
  "postgres://admin:xW2e2JpcTmWDiR@db.campuspay.ng:5432/prod";

// this one is fine, it reads from the environment
const jwtSecret = process.env.JWT_SECRET;

export async function charge(email, amount) {
  const API_TOKEN = "SOHmb6crdNF6tB0US5d6Dt3exIgCe48k";
  const gh = "ahp aZiC6VJrp2zR5ZPxrsTui8ZE5fro0vs0LK3a";
}
```

[Scan for secrets](#) [Load a leaky sample](#)

Results

3 possible secrets found (5 high, 0 medium)

- Stripe secret key** high
sk_L*****7a
Line 4, column 30. Roll the key in the Stripe dashboard.
- AWS access key ID** high
AKIA*****KB
Line 5, column 17. Deactivate the key in AWS IAM and create a new one.
- Database URL with password** high
post*****od
Line 6, column 13. Change the database password and update the connection string.
- Hard-coded API_TOKEN** high
SOHm*****8k
Line 12, column 22. Move this value into Hush and read it from the environment.
- GitHub token** high
ghp_*****3a
Line 13, column 15. Revoke the token in GitHub developer settings.

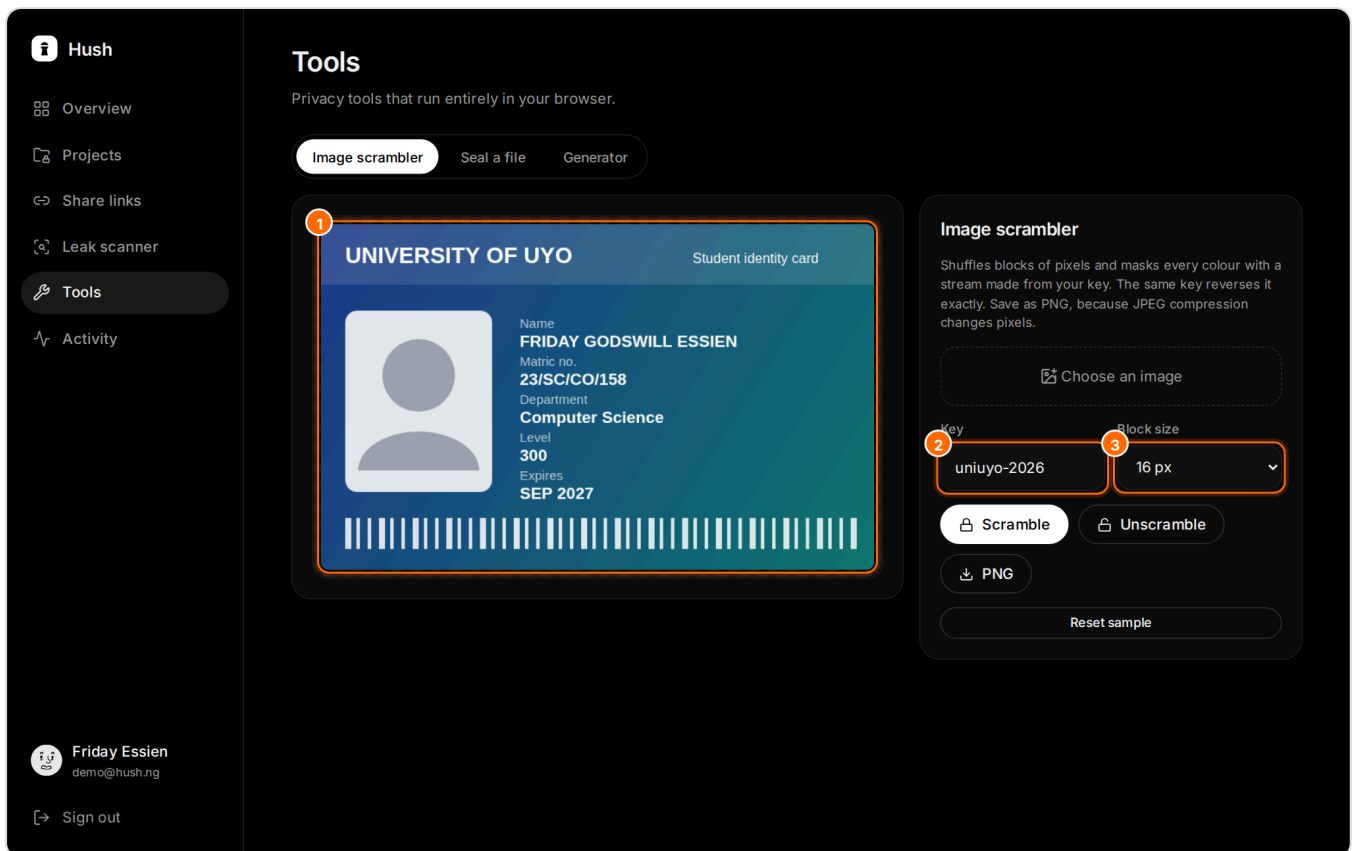
Friday Essien
demo@hush.ng

[Sign out](#)

- 1 Sample code with a payment key, an AWS key, a database URL and tokens.
- 2 Summary by severity.
- 3 Each finding shows the rule, a redacted match, the line and column, and what to do.

11. Image scrambler: original

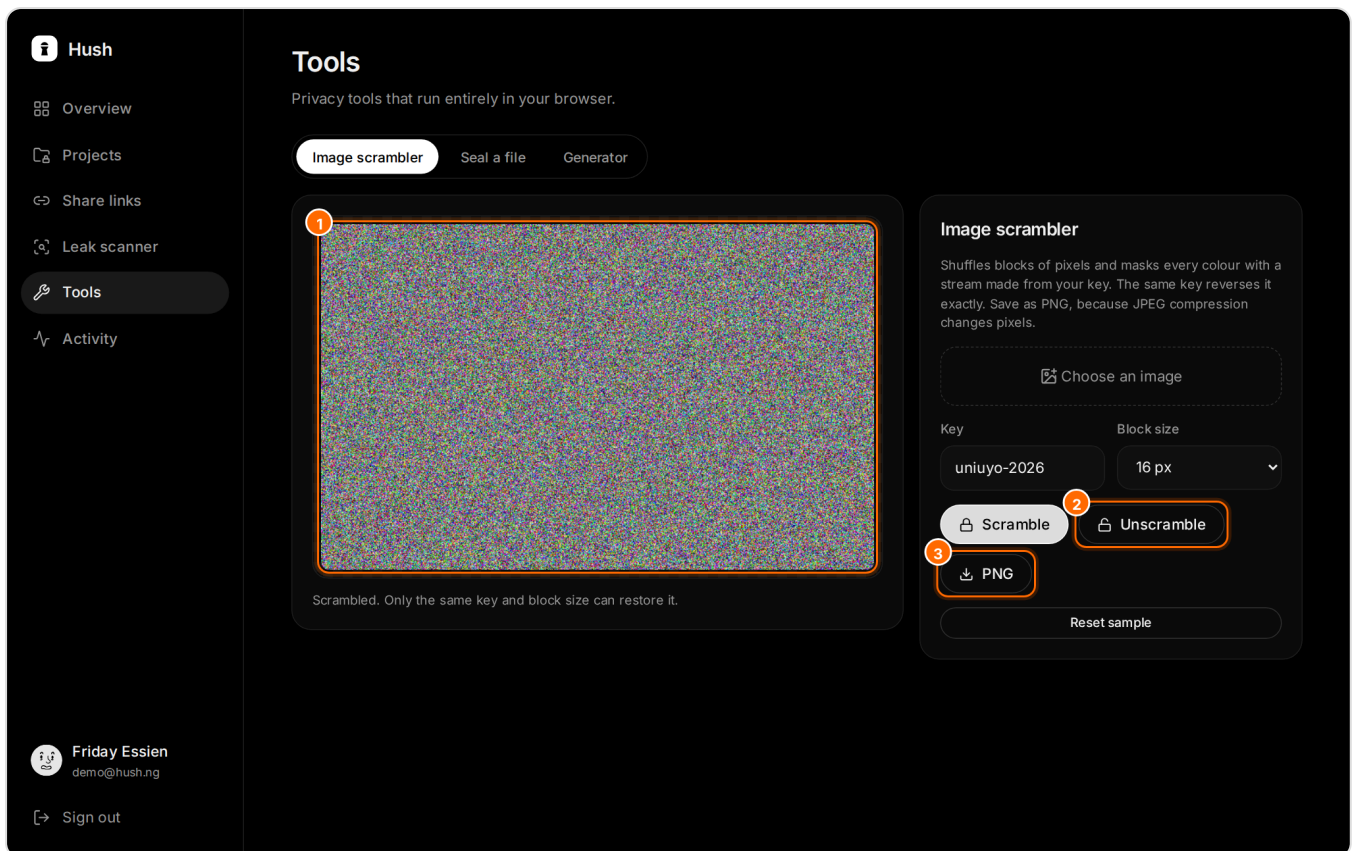
A sample student ID card is drawn on the canvas so the demo works without uploading a photo.



- 1 The image. Choose an image replaces it with your own file.
- 2 The key. Anyone who knows it can reverse the scramble.
- 3 Block size: smaller blocks hide more detail.

12. Image scrambler: scrambled

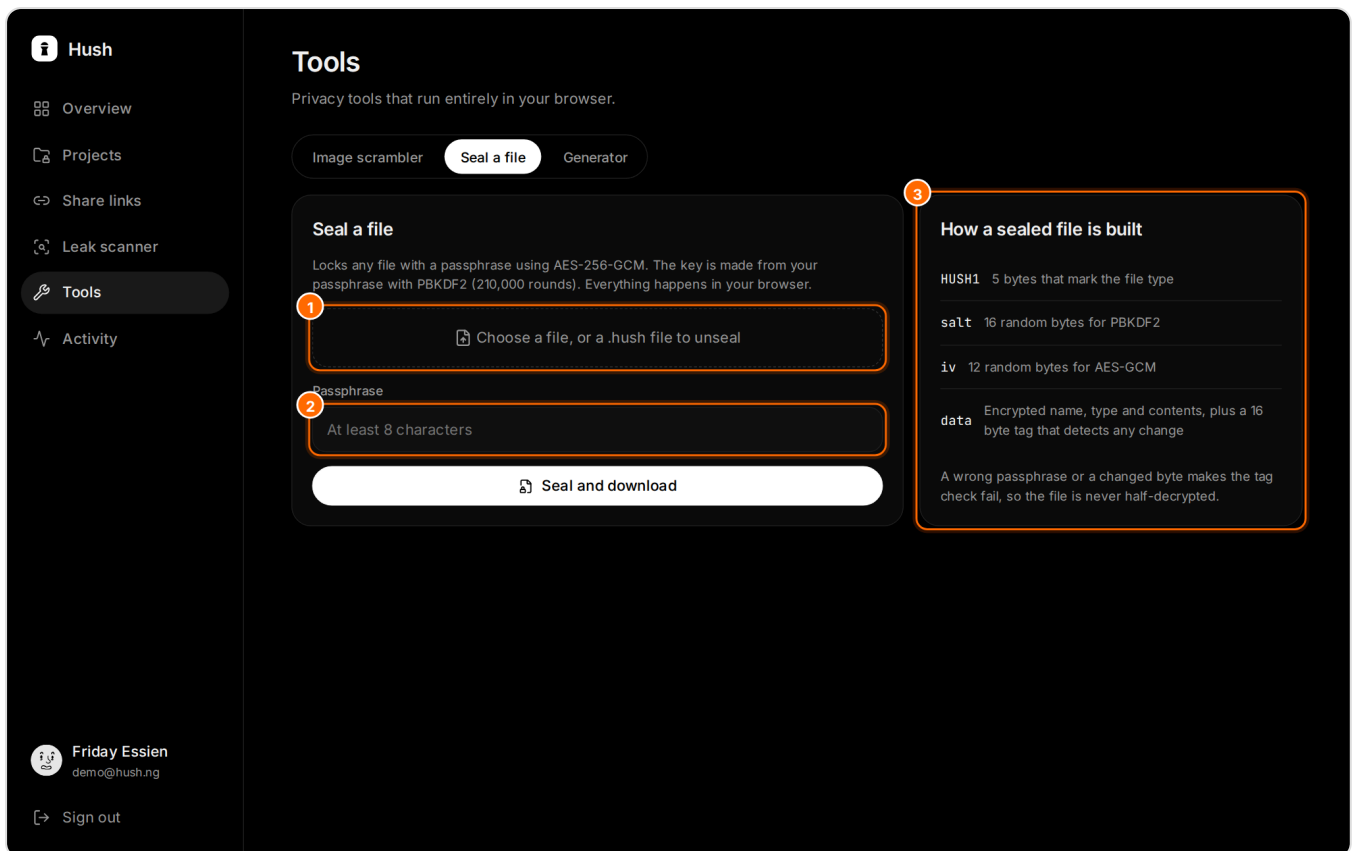
After Scramble the card is unreadable. Unscramble with the same key restores every pixel exactly.



- 1 Blocks shuffled and colours masked.
- 2 Reverses it. A wrong key leaves noise.
- 3 Download as PNG (lossless) so it can be restored later.

13. Seal a file

Encrypts any file with a passphrase in the browser and downloads a .hush file.



- 1 Pick any file, or a .hush file to unseal. The type is detected from its first bytes.
- 2 Passphrase, at least 8 characters. PBKDF2 turns it into an AES key.
- 3 The file format, explained for the audience.

14. Secret generator

Generates strong random values for new secrets.

The screenshot shows the Hush application interface. On the left is a sidebar with navigation links: Overview, Projects, Share links, Leak scanner, Tools (highlighted), and Activity. The main content area is titled 'Tools' and contains three buttons: 'Image scrambler', 'Seal a file', and 'Generator' (which is active). Below these buttons is the 'Secret generator' tool. It includes a description: 'Uses the browser's secure random source with rejection sampling, so every character is equally likely.' The tool has two input fields: 'Length' (set to 32) and 'Characters' (set to 'alphanumeric'). Below the 'Length' field, it shows '191 bits of entropy' and a 'Very strong' label. A 'Generate' button is positioned below these fields. The generated secret, 'fMJJQCS028R7apf2rvEbEXtTPc8bPIc', is displayed in a text box with a 'Copy' button next to it. Three orange numbered circles (1, 2, 3) are overlaid on the interface to highlight specific features: 1 points to the length and character set, 2 points to the entropy and strength label, and 3 points to the generated value and copy button.

- 1 Length and character set.
- 2 Entropy in bits and a strength label.
- 3 The generated value with a copy button.

15. Activity log

A record of every security-relevant action, filtered by type.

 Hush

Overview

Projects

Share links

Leak scanner

Tools

Activity

 Friday Essien
demo@hush.ng

Sign out

Activity

Every sign-in, reveal, export and share is recorded here.

1

All

Sign-ins

Secrets

Share links

Import and export

2

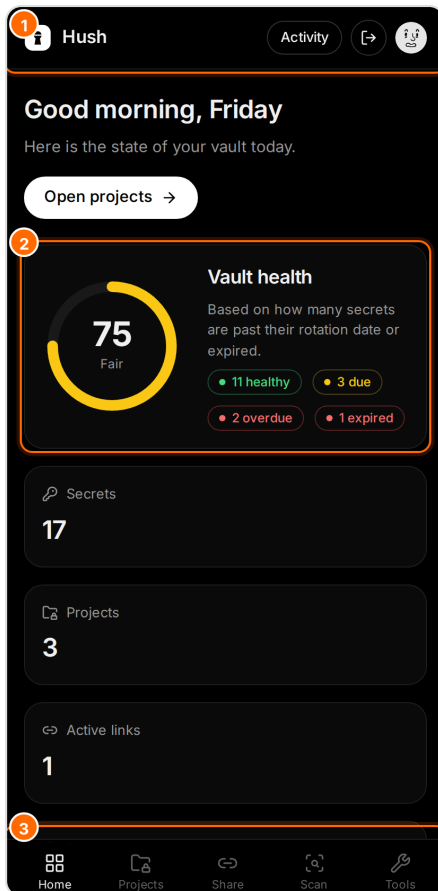
3 failed or blocked sign-in attempts in this list. Check the IP addresses.

EVENT	TARGET	IP ADDRESS	WHEN
3	Created share link DATABASE_URL (staging) 1 view(s), 24h	102.89.34.17	6 Oct, 09:24
	Share link opened DATABASE_URL (staging) view 1 of 1	127.0.0.1	6 Oct, 01:04
	Created share link DATABASE_URL (staging) 1 view(s), 24h	127.0.0.1	6 Oct, 01:04
	Revealed secret staging / DATABASE_URL	127.0.0.1	6 Oct, 01:04
	Revealed secret production / DATABASE_URL	127.0.0.1	6 Oct, 01:04
	Signed in Account	127.0.0.1	6 Oct, 01:04
	Signed in Account	127.0.0.1	6 Oct, 01:04
	Revoked share link RESEND_API_KEY (production)	102.89.34.17	5 Oct, 13:43
	Revealed secret staging / PAYSTACK_SECRET_KEY	102.89.34.17	4 Oct, 10:41
	Signed in Account	102.89.34.17	4 Oct, 10:41
	Failed sign-in Account	41.203.72.6	4 Oct, 00:30
	Failed sign-in Account	41.203.72.6	4 Oct, 00:30
	Failed sign-in Account	41.203.72.6	4 Oct, 00:30
	Exported .env Campus Pay API / development 2 keys	102.89.34.17	1 Oct, 09:59
	Rotated secret production / DATABASE_URL now version 2	102.89.34.17	29 Sept, 11:19
	Share link opened MAPBOX_TOKEN (production) view 1 of 1	105.112.24.9	27 Sept, 18:54
	Created share link MAPBOX_TOKEN (production) 1 view(s), 7d	102.89.34.17	27 Sept, 17:51
	Revealed secret production / DATABASE_URL	102.89.34.17	26 Sept, 12:43
	Imported .env Hostel Finder / production 4 added, 0 updated, 0 skipped	102.89.34.17	24 Sept, 15:06
	Created project Portfolio Site	102.89.34.17	23 Sept, 10:58
	Signed in Account	102.89.34.17	23 Sept, 10:58

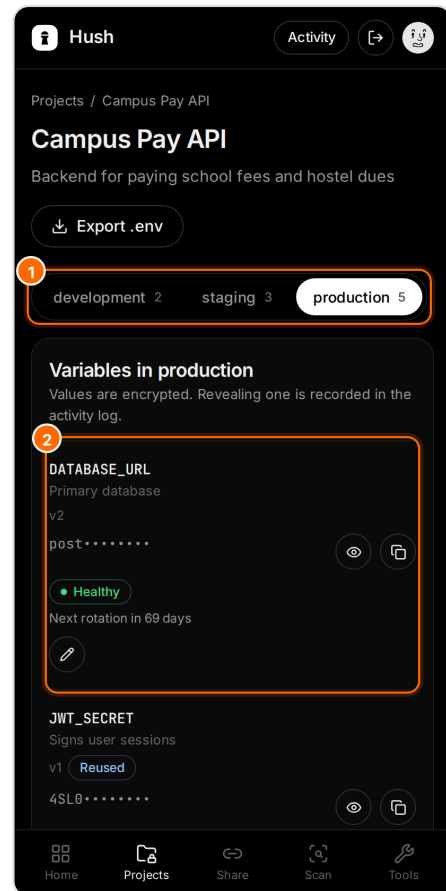
- 1 Filter by sign-ins, secrets, share links or import and export.
- 2 Failed and blocked sign-ins are highlighted.
- 3 Each event shows what happened, the target, the IP address and the time.

5. Mobile view

Every page works at phone width with no sideways scrolling. CSS grids use `minmax(0, 1fr)` so long keys and values wrap instead of widening the page. An automated test checks eight pages on a Pixel 7 screen size.



Mobile: overview



Mobile: project

Mobile: overview

On a phone the sidebar becomes a top bar and a bottom tab bar.

- 1 Top bar with activity, sign out and avatar.
- 2 Cards stack in one column.
- 3 Bottom tab bar within thumb reach.

Mobile: project

Table rows turn into stacked cards so nothing scrolls sideways.

- 1 Environment tabs.
- 2 Each secret as a card: key, masked value, reveal and copy, status.

6. Running locally

Requirements: Node.js 20.9 or newer and PostgreSQL 14 or newer.

```
service postgresql start
createdb hush
cp .env.example .env          # set DATABASE_URL and SESSION_SECRET
npm install
npm run db:migrate            # apply SQL migrations
npm run db:seed                # demo data dated relative to today
npm run dev                    # http://localhost:3000
```

Sign in with `demo@hush.ng` and `demo1234`. To freeze the date for a demo, add `HUSH_TODAY=2026-10-06` to `.env`.

Script	What it does
<code>npm run dev / build / start</code>	Develop, build and serve the app
<code>npm run db:generate</code>	Create a new SQL migration after changing <code>src/db/schema.ts</code>
<code>npm run db:migrate</code>	Apply migrations (creates the database first if it is missing)
<code>npm run db:seed</code>	Reset and seed demo data; <code>db:seed:if-empty</code> seeds only an empty database
<code>npm test</code>	Unit and integration tests (needs the <code>hush_test</code> database)
<code>npm run test:e2e</code>	Playwright tests on desktop and mobile (needs a build and the <code>hush_e2e</code> database)
<code>npm run docs</code>	Rebuild this PDF

7. Testing

Tests run at three levels. All use a fixed date, and none depend on the network.

Suite	Tests	What it checks
Unit (Vitest)	61	Dates, encryption and AAD, tamper detection, rotation and health score, .env parsing and writing, scanner rules, share states, lockout, scrambling round trip, file sealing, generator bias, validation, session tokens
Integration (Vitest, real PostgreSQL)	18	Seed data, ciphertext only in the database, user scoping, create, reveal, rotate, reuse detection, import and export, one-time links including two simultaneous opens, lockout, sign-up
End-to-end desktop (Playwright)	17	Sign in and up, errors, redirects, project statuses, reveal and audit, add and rotate secrets, import and export, share link opened once, scanner, scrambler, sealing, generator, 404 for other users, health check
End-to-end mobile (Playwright, Pixel 7)	2	No sideways scrolling on eight pages; bottom navigation and revealing a secret
Total	98	All passing

```
createdb hush_test && createdb hush_e2e
npm test                # 79 unit and integration tests
npm run build && npm run test:e2e # 19 browser tests
```

Selected test cases

Case	Expected	Result
Sign in with the pre-filled demo account	Dashboard greets Friday	Pass
Copy a production ciphertext into another row	Decryption fails	Pass
Open a one-time link twice at the same moment	Exactly one succeeds	Pass
Six sign-ins with a wrong password, then the right one	Locked for 15 minutes	Pass
Import a .env with a line missing =	Inline error: Line 4: Missing = sign	Pass
Unscramble with a wrong key	Image stays scrambled	Pass
Open another user's project by URL	404 page	Pass
View eight pages on a phone	No horizontal scrolling	Pass

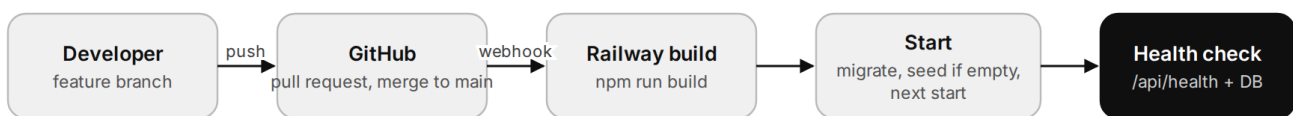
8. Deployment

Hush runs on Railway in the project *school-projects* as the service *hush*, deployed from the `main` branch of the GitHub repository. Every merge to `main` triggers a new build.

Setting	Value
Build	<code>npm run build</code>
Start	<code>npm run db:migrate</code> , then <code>npm run db:seed:if-empty</code> , then <code>next start -H 0.0.0.0</code>
Health check	<code>/api/health</code> runs <code>select 1</code> on the database and returns <code>{"status":"ok"}</code>
DATABASE_URL	Reference to the Postgres service in the project, database name <code>hush</code>
SESSION_SECRET	48 random characters, set once in Railway variables
NODE_ENV	<code>production</code>
Public URL	<code>https://hush-production-70be.up.railway.app</code>

The Railway project already had ten database volumes, the most it allows, so Hush uses its own database named `hush` on an existing Postgres server instead of a new volume. The migrate script creates the database on first start if it does not exist.

Changing `SESSION_SECRET` signs everyone out and makes stored values unreadable, because the vault key is derived from it. To rotate the session key without losing data, set `VAULT_KEY` to the old secret first.



Traffic moves to the new version only after the health check returns OK.

Figure 7. Deployment pipeline

9. Five-minute presentation script

Sign in on the live site before you start and keep a second browser window (or a phone) ready for the share link.

Time	What to say and do
0:00 to 0:40	Problem. "Developers keep passwords and API keys in .env files and share them on WhatsApp. Keys never get rotated and nobody knows who saw what." Show the landing page and the six features.
0:40 to 1:30	Sign in (the demo login is pre-filled). On the overview, point at the health score of 75, then the Needs attention list: an expired SMTP password, an overdue Paystack key and keys due this week. Point at Reused values: 2.
1:30 to 2:30	Open Campus Pay API. Reveal DATABASE_URL and say the database only holds ciphertext. Point at the Reused badge on JWT_SECRET and the Expired badge. Switch to staging and show the missing-keys panel. Click Export .env.
2:30 to 3:15	Open Hostel Finder, edit ADMIN_PASSWORD, use the generator value as the new password and save. Show it turn Healthy and the version go up.
3:15 to 4:00	Share links: create a one-time link for a staging key, open it on the phone, press Reveal, then refresh to show it no longer works. Back on Activity, show the Share link opened event and the three failed sign-ins from 41.203.72.6.
4:00 to 4:40	Leak scanner: Load a leaky sample and show the findings. Tools: scramble the student ID card, unscramble with a wrong key (still noise), then the right key (restored).
4:40 to 5:00	Close: AES-256-GCM with bound AAD, hashed share links, lockout, 98 automated tests, deployed on Railway. Invite questions.

Likely questions

Question	Short answer
What if the database is stolen?	The attacker gets ciphertext and hashes. Without SESSION_SECRET or VAULT_KEY, which lives only in Railway, the values cannot be read.
Can the admin of the server read secrets?	Yes, the server can decrypt to show values. A future version would encrypt in the browser with per-user keys so the server never sees plaintext.
Is the image scrambler real encryption?	No. It keeps the file a viewable image. Use Seal a file for confidential data.
Why not just use a password manager?	Hush is built around projects and environments: rotation periods, .env import and export, differences between environments and reuse detection.